Lab18_N.RUKMINI_2403A52402

Sequence-to-Sequence Tasks — Using Hugging Face Pipelines for Summarization

STEP 1 — Install and Import Libraries

```
!pip install transformers torch

from transformers import pipeline
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.0.0)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cpu)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.25.2)
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (1
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (26.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from transformers) (0.24.0)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.3.2 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transform
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->h
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,>=1.
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,>=1.
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->h
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer-
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3
```

Explanation

Pretrained libraries like Hugging Face Transformers are used because:

Training summarization models from scratch needs huge data and time

Pretrained models are already trained on large datasets

Advantages of Transformer Models

Better context understanding (uses attention mechanism)

Handles long text better than traditional methods

Generates human-like summaries (abstractive)

STEP 2 — Load Summarization Model

```
summarizer = pipeline("text-generation", model="facebook/bart-large-cnn")
print("Model loaded successfully!")
```

```
Loading weights: 100%                    316/316 [00:00<00:00, 659.22it/s, Materializing param=model.decoder.layers.11.self_attn_layer_norm.weight]
BartForCausalLM LOAD REPORT from: facebook/bart-large-cnn
Key                                                        | Status     |  |
-----------------------------------------------------------+------------+--+-
model.encoder.layers.{0...11}.self_attn.out_proj.bias      | UNEXPECTED |  |
model.encoder.layers.{0...11}.fc2.weight                   | UNEXPECTED |  |
model.encoder.layers.{0...11}.self_attn_layer_norm.bias    | UNEXPECTED |  |
model.encoder.layers.{0...11}.self_attn.q_proj.bias        | UNEXPECTED |  |
model.encoder.layers.{0...11}.fc2.bias                     | UNEXPECTED |  |
model.encoder.layers.{0...11}.self_attn.v_proj.bias        | UNEXPECTED |  |
model.encoder.layers.{0...11}.final_layer_norm.bias        | UNEXPECTED |  |
model.encoder.layers.{0...11}.self_attn.k_proj.weight      | UNEXPECTED |  |
model.encoder.layers.{0...11}.self_attn_layer_norm.weight  | UNEXPECTED |  |
model.encoder.layers.{0...11}.self_attn.k_proj.bias        | UNEXPECTED |  |
model.encoder.layers.{0...11}.self_attn.q_proj.weight      | UNEXPECTED |  |
model.encoder.layers.{0...11}.final_layer_norm.weight      | UNEXPECTED |  |
model.encoder.layers.{0...11}.self_attn.v_proj.weight      | UNEXPECTED |  |
model.encoder.layers.{0...11}.fc1.weight                   | UNEXPECTED |  |
model.encoder.layers.{0...11}.fc1.bias                     | UNEXPECTED |  |
model.encoder.layers.{0...11}.self_attn.out_proj.weight    | UNEXPECTED |  |
model.encoder.layernorm_embedding.bias                     | UNEXPECTED |  |
model.encoder.layernorm_embedding.weight                   | UNEXPECTED |  |
model.encoder.embed_positions.weight                       | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Model loaded successfully!
```

Explanation

Pipeline = ready-made function for NLP tasks

Pretrained model = already trained summarization model

facebook/bart-large-cnn:

Best for news & long text summarization

Produces high-quality abstractive summaries

STEP 3 — Prepare Input Texts

```
texts = [
"""Artificial Intelligence (AI) is transforming industries by enabling machines
to learn from data and make decisions. AI is used in healthcare, education,
finance, and transportation. However, challenges such as data privacy and
ethical concerns must be addressed.""",

"""The Indian economy has shown steady growth over the past decade.
Government reforms, digitalization, and infrastructure development
have contributed significantly. However, inflation and unemployment
remain key challenges.""",

"""Once upon a time, there was a brave knight who set out on a journey
to rescue a princess. Along the way, he faced many obstacles and learned
valuable lessons about courage and friendship.""",

"""In software engineering, testing is a crucial phase. It ensures that
applications function correctly and meet user requirements. Different
types of testing include unit testing, integration testing, and system testing.""",

"""Hey, did you complete the assignment? Not yet, I was busy yesterday.
We should finish it today before the deadline. Yes, let’s work together and submit it on time."""
]
```

Explanation

Different text types help test model performance

Challenges:

Technical text → complex terms

Conversation → informal language

Stories → narrative flow

STEP 4 — Generate Summaries

```
for i, text in enumerate(texts):
    summary = summarizer(text, max_length=50, min_length=20, do_sample=False)
```

```
    print(f"\nText {i+1} Summary:")
    print(summary[0]['generated_text'])
```

```
Text 1 Summary:
Artificial Intelligence (AI) is transforming industries by enabling machines
to learn from data and make decisions. AI is used in healthcare, education,
finance, and transportation. However, challenges such as data privacy and
ethical concerns must be addressed.

Text 2 Summary:
The Indian economy has shown steady growth over the past decade.
Government reforms, digitalization, and infrastructure development
have contributed significantly. However, inflation and unemployment
remain key challenges. actually actually actuallyeray actually actually

Text 3 Summary:
Once upon a time, there was a brave knight who set out on a journey
to rescue a princess. Along the way, he faced many obstacles and learned
valuable lessons about courage and friendship.AS only only only always

Text 4 Summary:
In software engineering, testing is a crucial phase. It ensures that
applications function correctly and meet user requirements. Different
types of testing include unit testing, integration testing, and system testing. this this this one this still

Text 5 Summary:
Hey, did you complete the assignment? Not yet, I was busy yesterday.
We should finish it today before the deadline. Yes, let’s work together and submit it on time. one Only had only only last M
```

Explanation

Model understands key points and removes unnecessary details

Patterns observed:

Keeps important sentences

Shortens long explanations

Simplifies language

STEP 5 — Control Summary Length

```
text = texts[0]

short_summary = summarizer(text, max_length=30, min_length=10, do_sample=False)
medium_summary = summarizer(text, max_length=50, min_length=20, do_sample=False)
long_summary = summarizer(text, max_length=80, min_length=30, do_sample=False)

print("Short Summary:\n", short_summary[0]['generated_text'])
print("\nMedium Summary:\n", medium_summary[0]['generated_text'])
print("\nLong Summary:\n", long_summary[0]['generated_text'])
```

```
Short Summary:
 Artificial Intelligence (AI) is transforming industries by enabling machines
to learn from data and make decisions. AI is used in healthcare, education,
finance, and transportation. However, challenges such as data privacy and
ethical concerns must be addressed.

Medium Summary:
 Artificial Intelligence (AI) is transforming industries by enabling machines
to learn from data and make decisions. AI is used in healthcare, education,
finance, and transportation. However, challenges such as data privacy and
ethical concerns must be addressed.

Long Summary:
 Artificial Intelligence (AI) is transforming industries by enabling machines
to learn from data and make decisions. AI is used in healthcare, education,
finance, and transportation. However, challenges such as data privacy and
ethical concerns must be addressed. had had had only only only still had only had had the had had both still had still had h
```

Explanation

max_length → maximum words in summary

min_length → minimum words

do_sample → randomness

- Observation

Short → less detail

Medium → balanced

Long → more information

👉 Trade-off:

Short = concise but may lose details

Long = informative but less concise

STEP 6 — Comparison of Text Types (Analysis)

```
#News texts are summarized very accurately because models are trained on similar data

#Technical texts lose some domain-specific terms

#Narrative texts retain main storyline but lose emotional depth

#Conversational texts are harder to summarize due to informal structure

#Information coverage is highest in structured texts

#Coherence is generally good but may slightly drop in long inputs

#Important details are sometimes skipped in shorter summaries

#Domain-specific words may be simplified or removed

#Model performs best on formal and well-structured text

#Informal and mixed-style text gives less accurate summaries
```

STEP 7 — Very Long Text Test

```
long_text = """(Paste a 400+ word paragraph here)"""

summary = summarizer(long_text, max_length=100, min_length=40, do_sample=False)
print(summary[0]['generated_text'])
```

```
(Paste a 400+ word paragraph here) Dyeve Bare Bare Bareevereverever En Bare Barece Laer Barealdteerteer Bare Bareteteteerert
```

Observations

Important points are mostly retained

Some details are lost

Structure becomes more compressed

Limitation: input length restriction

STEP 8 — Lab Report (Write This)

Objective

To perform text summarization using pretrained Transformer models.

🔹 Dataset Description

The dataset consists of 5–8 text passages collected from news articles, blogs, technical documents, and conversations. Each passage contains 150–400 words and represents different writing styles.

🔹 Installation

Installed transformers and torch using pip

Used Hugging Face pipeline for summarization

🔹 Outputs

Generated summaries for all texts

Compared short, medium, and long summaries

🔹 Parameter Comparison

max_length controls size of summary

min_length ensures minimum detail

do_sample introduces randomness

- Analysis

Model performs best on structured text

Less accurate on conversational text

Trade-off between brevity and detail

- Conclusion

Transformer-based summarization is effective and fast. However, it may lose important details and struggles with informal text.

Theory Answers (Important for Exam)

1. What is text summarization?

Text summarization is the process of converting long text into a shorter version while preserving key information.

2. Extractive vs Abstractive

Extractive → picks sentences from text

Abstractive → generates new sentences (like humans)

3. Sequence-to-Sequence Model

A model that converts input sequence → output sequence (e.g., text → summary)

4. Encoder–Decoder Architecture

Encoder → understands input

Decoder → generates output

5. Hugging Face Pipeline

A simple API to perform NLP tasks like summarization, translation, etc.

6. Why pretrained models?

Saves time

No need for large data

High accuracy

7. Limitations

May lose important details

Struggles with long text

Less accurate for informal language

8. Real-life Applications

News summarization

Document summarization (legal, academic)